COSC-4117EL: Assignment 2 Report

Group Number: 14

NAME	STUDENT#	EMAIL
Jai Dey	0477282	jdey@laurentian.ca
Seyyedeh Shima Nejati	0461677	snejati@laurentian.ca

Repository URL: https://github.com/deyjai/reinforcement-learning-gridworld

---

Introduction:

We will be using reinforcement learning to train an agent to navigate a warehouse. This agent will be performing 3 tasks within the warehouse: picking up the package, delivering the package, and returning to the starting square.

Q-learning allows us to automate the path for these 3 tasks. This is a type of reinforcement learning where we train our agent to take the best actions to take in each situation by interacting with the environment and receiving rewards. We will also be implementing a second reinforcement learning algorithm, which is similar to Q-learning but it incorporates an exploration bonus, which encourages the robot to try less-visited actions to find better paths.

Environment and State Space

Our virtual warehouse is modeled as a square grid. Each cell in the grid represents one of the following: An open space, a wall, a congestion zone, or a special location such as the pickup station, packing station, or charging dock..

Within the virtual warehouse, our agent can move in four directions: up, down, left, or right. And the state of our agent is represented as a tuple as following:

(row, column, has_package, delivered)

Naturally, *row* and *column* are integers that represent our agent's current location in the virtual warehouse. The other two parameters, *has_package* and *delivered* are boolean values that represent whether our agent is carrying a package or if the package has been delivered respectively. This is used to track the completion state of our 3 tasks, and to determine which task the agent is currently performing.

How the Q-Learning Algorithm Works

Firstly, we initialize a Q-table to store our agent's expected reward for every possible state-action pair. Then we train our agent by forcing it to navigate the virtual warehouse and repeatedly update its Q-table.

At each step, our agent selects an action based on the value of *epsilon*. Epsilon represents the probability for the agent to choose a random action instead of the best-known action (from the Q-table). Because ε starts high, the agent will mostly choose random actions during this first episode. It then attempts to move in the chosen direction, and will update the Q-table based on the rewards of the new cell that it moves into.

This process continues iteratively until either our agent completes all tasks or it reaches the maximum number of steps. By the end of the first episode, the Q-table is no longer all zeros, and the agent has started forming basic estimates of good and bad actions. Moreover, at the end of each episode, the value of *epsilon* is slightly decreased as it is multiplied by a factor of its *decay rate*.

We trained our agent over 2000 episodes and the resultant Q-table is what allows us to extract our *greedy policy*. This represents the path where during each step our agent makes the action that corresponds with the highest Q-value.

Q-Learning with Exploration Bonus

This algorithm is very similar to the above Q-Learning algorithm, except that we implemented a mechanism that uses an exploration bonus to guide the agent's behavior.

Now instead of having just one Q-table, we maintain a second table N(s,a). This table counts how many times the agent has taken a specific action in a given state. When selecting an action, instead of choosing randomly, we modify the Q-value by adding a bonus term:

$$Q_{Value\ Modified} = Q(s, a) + k/\sqrt{N(s, a) + 1}$$

This means that when an action is rarely chosen, its modified Q-value receives a high bonus and is thus more likely to be selected. As the agent continues to explore and the visit count increases, the bonus decreases, allowing the agent to gradually rely more on the learned Q-values.

Essentially, this encourages exploration of less-visited actions, and hence this leads to more efficient training in the earlier stages of the training process.

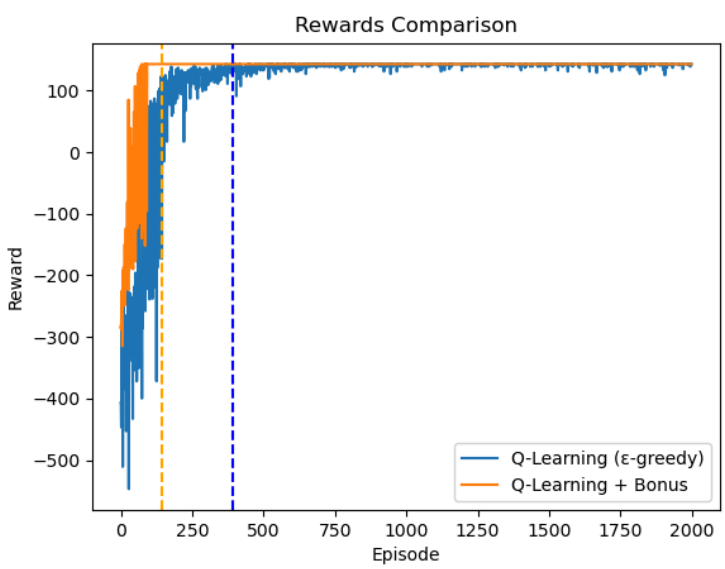
Base Hyperparameters

Below were the default hyperparameters that we used during training of both Q-learning algorithms.

Parameter	Value	Description
ALPHA	0.1	Learning rate – controls how fast Q-values are updated
GAMMA	0.99	Discount factor – balances immediate vs future rewards
EPSILON_START	1.0	Initial exploration rate for ε-greedy Q-Learning
EPSILON_MIN	0.01	Minimum ε to ensure some exploration
EPSILON_DECAY	0.995	Rate at which ε decays per episode
EPISODES	2000	Number of training episodes
MAX_STEPS	120	Maximum moves per episode
K	5.0	Exploration bonus constant for the bonus algorithm

Experimental Results and Discussion:

Figure 1: Rewards for Each Q-Learning Algorithm



The above figure plots the rewards vs episode, using a warehouse environment of seed 42. The dotted lines represent the point of convergence for each algorithm respectively. From this we can see that the reward per episode was significantly higher everywhere for our *Q-learning + bonus* algorithm. At all stages the reward was either higher or equivalent to the *Q-Learning (ε-greedy)* algorithm.

Figure 2: Cumulative Number of Successes

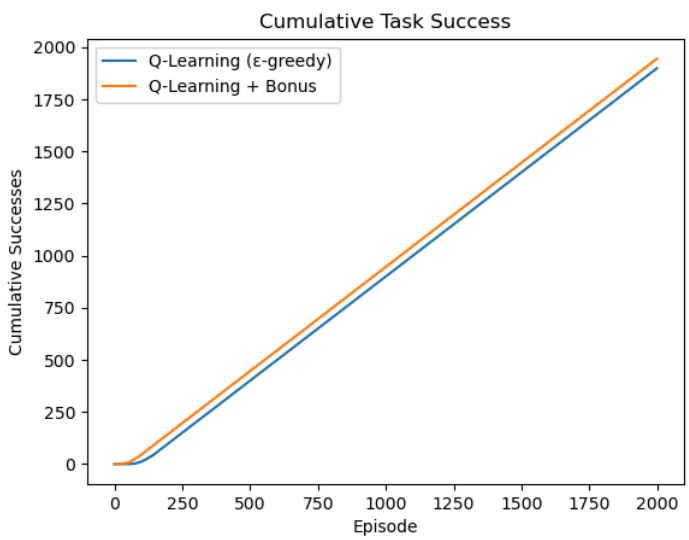


Figure 2, above, was plotted using seed 42. As expected, the cumulative number of task successes of the *Q-learning + bonus* algorithm was higher than the *Q-Learning (ε-greedy)* algorithm during all points of training.

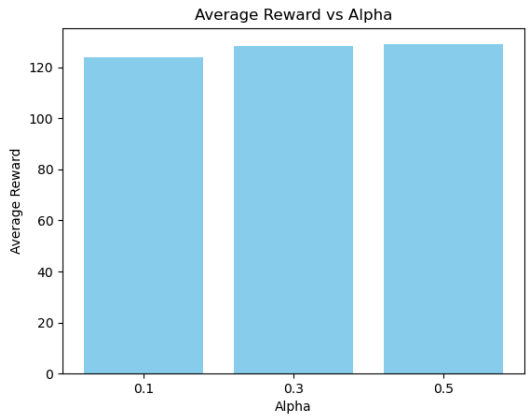
Figure 3: Convergence Episodes and Average Metrics (seed 42)

```
Approximate convergence episodes (rewards):
  Q-Learning (ε-greedy): 391
  Q-Learning + Bonus   : 142

Overall metrics across all episodes:
  Q-Learning (ε-greedy): Avg reward = 115.48, Success rate = 94.95%
  Q-Learning + Bonus   : Avg reward = 133.44, Success rate = 97.25%
```

Note that the convergence threshold was calculated with tol=0.01. Hence, the change in the average reward between the previous window and next window is less than 1% of the maximum reward seen so far.

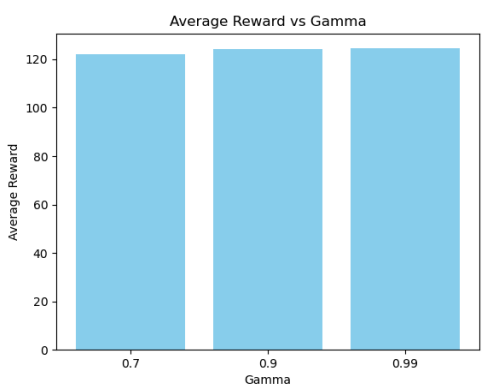
Figure 4.1: Average Rewards vs Alpha




```
Average Rewards vs Alpha:
Alpha | Average Reward
----------------------
0.10  | 124.31
0.30  | 128.30
0.50  | 128.86
```

As shown by Figure 4.1, we can see that the average reward shifts as α increases. Once α moves past 0.1, the agent updates its value estimates aggressively enough that the performance bump is immediate and stable. The jump from 0.3 to 0.5 is marginal, which suggests we're already in the regime where learning-rate sensitivity flattens out. The true peak can potentially be around the value of α = 0.55.

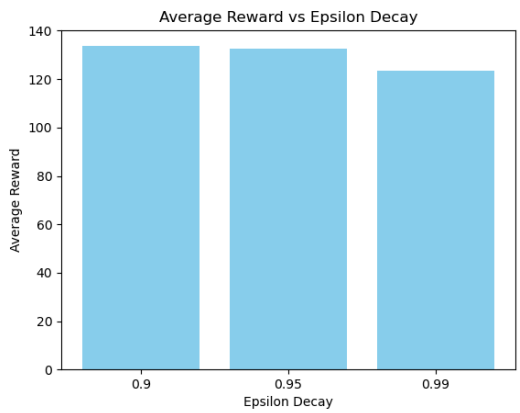
Figure 4.2: Average Rewards vs Gamma




```
Average Rewards vs Gamma:
Gamma | Average Reward
----------------------
0.70  | 122.22
0.90  | 124.27
0.99  | 124.51
```

Figure 4.2 shows the same sweep but for γ. Notably, the differences are smaller here. This is because γ mostly affects long-term credit assignment, and this environment doesn't seem to reward deep lookahead. However, pushing γ toward 0.99 still gives a slight improvement, indicating that the agent benefits from carrying more future reward into its updates, just not dramatically so.

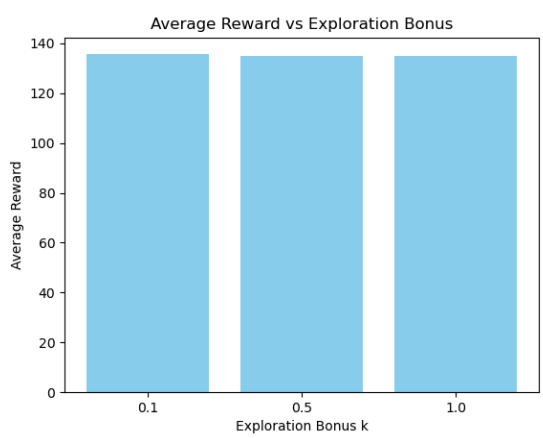
Figure 4.3: Average Rewards vs Epsilon Decay




```
Average Rewards vs Epsilon Decay:
Epsilon Decay | Average Reward
------------------------------
0.90          | 133.65
0.95          | 132.63
0.99          | 123.52
```

Figure 4.3 shows how the average reward shifts as the ε-decay rate changes. According to the above table, we can see that the fastest decay rate that we tested (0.90) gives the best performance. As it turns out, committing to the Q-table earlier works better in our environments. With a very slow decay rate, the agent spends too long in exploratory mode. I would estimate the optimal epsilon decay value to be around 0.85.

Figure 4.4: Average Rewards vs Exploration Bonus




```
Average Rewards vs Exploration Bonus (k):
k     | Average Reward
--------------------
0.10  | 135.63
0.50  | 135.13
1.00  | 135.08
```

As shown above, Figure 4.4 looks at the exploration-bonus coefficient *k*. The differences here are very minimal compared to Figures 4.1-4.3. This is a good sign, because it suggests that the bonus term (being the denominator) is doing its job even at the lowest setting. Hence, increasing *k* does not change the average reward much because the agent already gets enough incentive to visit underexplored states.

Figure 5: Testing Results Across Multiple Seeds

```
Q-Learning (ε-greedy):
  Seed 42: Avg reward = 115.71, Success rate = 95.10%
  Seed 123: Avg reward = 82.25, Success rate = 89.90%
  Seed 999: Avg reward = 98.50, Success rate = 92.95%
Q-Learning + Bonus:
  Seed 42: Avg reward = 133.44, Success rate = 97.25%
  Seed 123: Avg reward = 101.49, Success rate = 93.40%
  Seed 999: Avg reward = 115.95, Success rate = 94.85%
```

From Figure 5, we can see that our *Q-Learning + bonus* algorithm is very consistently outperforming the *Q-Learning (ε-greedy)* algorithm. The average reward and success rates, when using the modified algorithm are all higher than the *Q-Learning (ε-greedy)*. Hence, both algorithms can be considered robust across multiple seeds.

Conclusion:

In conclusion, the *Q-Learning + bonus algorithm* consistently performs better than the *Q-Learning (ε-greedy)* algorithm. This is because exploring bonuses allows the *Q-Learning + bonus* algorithm to ensure the agent visits other unexplored state spaces much earlier than what it would normally be visiting. This ensures that the agent converges at a higher rate and completes more tasks. Although the normal Q-Learning algorithm could also find a way of performing effectively, it is more likely to get trapped on suboptimal paths because of less frequent state visits.

Resources:

1. https://www.researchgate.net/profile/Michel-Tokic/publication/221562563_Adaptive_e-Greedy_Exploration_in_Reinforcement_Learning_Based_on_Value_Differences/links/00b4953429c74d0591000000/Adaptive-e-Greedy-Exploration-in-Reinforcement-Learning-Based-on-Value-Differences.pdf

2. https://www.researchgate.net/profile/Shah-Asif-Bashir-2/publication/357127412_Adaptive_epsilon-Greedy_Exploration_for_Finite_Systems/links/61bca77a4b318a6970eac369/Adaptive-epsilon-Greedy-Exploration-for-Finite-Systems.pdf

3. https://chatgpt.com/share/69d09a83-5004-832f-9d88-836e281628a1

4. https://chatgpt.com/share/69d09b2b-cee4-8331-9cae-66b59ace9525